

A semi-independent policies training method with shared representation for heterogeneous multi-agents reinforcement learning

- Parameter sharing works well for homogeneous agents, where one policy is trained on everyone's trajectories.
- For heterogeneous agents (different observation/action spaces, roles, goals), it breaks down:
 - Zero-padding inputs/outputs adds redundant information and misleads early policies.
 - Sharing everything homogenizes behavior; sharing nothing converges slowly and wastes experience.
- Goal: balance experience sharing against agent specialization.
- Inspiration: the brain shares abstract knowledge across tasks.

Method (STSR)

Semi-independent Training with Shared Representation

- Common representation: an encoder-decoder (GRU-based) forward model predicts the next observation and reward from the current observation and action. Its latent embedding gives all agent types a unified input/output format.
- Two-level parameter sharing:
 - Agents of the same type share all parameters.
 - Different types share lower layers (hard parameter sharing, borrowed from multi-task learning) and keep type specific output layers.
- The shared layer is initialized from the representation, then updated by RL, while the predictor keeps training in parallel.

Intrinsic Rewards

- Total reward: $r_{\text{tot}} = r + \lambda_e \cdot r_e + \lambda_d \cdot r_d$
- Exploration reward (r_e): the forward model's prediction error, weighted by mixer weights. It gives a bonus in novel, low-density states, and its weight decays over training.
- Diversity/identification reward (r_d): pushes representations of same-type agents together and different-type agents apart.
- Both rewards reuse the same predictor, so they cost no extra learning machinery.

Experiments and Results

- Benchmark: SMAC (StarCraft II micromanagement), 14 scenarios, 4 random seeds, median reported.
- Baselines: VDN, QMIX, QPLEX, ROMA, RODE, CDS.
- Best on 3s5z_vs_3s6z and MMM2 (both heterogeneous).
- Second on 3s5z and 27m_vs_30m.
- Ablations: all components help. Removing the representation's later update hurts most, while hard parameter sharing mainly speeds up training rather than raising final performance.
- Transfer: the policy trained on 3s5z_vs_3s6z was tested on larger unseen maps with no retraining, and did best when both sides scaled up.

Limitations and Takeaways

- Limitations- author's:
 - Weaker on homogeneous settings than RODE, ROMA and CDS.
 - Struggles when one type has many agents (e.g., bane_vs_bane's 20 Zerglings).
 - Grouping is by unit type only, so it needs type labels, and random clustering sometimes beat it on homogeneous maps.
- Takeaway: a shared learned representation plus type-level parameter sharing gives faster convergence and better transfer for heterogeneous teams.
- Future work: better parameter-sharing groups and clustering.